MACHINE LEARNING

HOMEWORK – 2

Ronitt Mehra, rm4084

Hunter Wilder, hdw2112

Question – 1

A.

The first things needed to build a generative model are the marginal distributions of the output values Y (in this case Y = (y_i) = (0,1,2)) and then the conditional distributions of the input parameters 'x' give some output 'y' value, so we calculated these quantities. Getting marginal y was easy, just add up all the y of a particular value and then divide by the total. And then for the conditional x distribution, we assumed a normal distribution shape and so for each type of output of y we clumped all of their feature values together by index and then calculated the mean and variance of each feature for each output which then allowed us to build a continuous normal distraction representative of all of the conditional feature values. Since our classifier is comparing 2 features, we had to use the 2D Gaussian form:

$$(\text{Gaussian}) = Ae^{\frac{(x^2-\mu_1^2)}{\sigma_1}} * e^{\frac{(x^2-\mu_2^2)}{\sigma_2}}$$

and then from there all we needed to do is to build a classifier was to take in input from a new item and then plug in its feature value to each gaussian given by the conditional probabilities and pick the one that gives the highest value because that corresponds to the most likely chance of being right. and then to optimize the feature choice of 2 features you should split the data in the training set to get the needed statistic of the generative distribution and then test the feature combinations on the other half of the data. to test the feature combinations, we just test every possible feature combination which shouldn't be too hard since the data set is not too big. And then finally test your classifier on the test data set.

```
#PROBLEM 1 CODE

import pickle as p
import numpy as np
import copy as cp
import matplotlib.pyplot as plt
from matplotlib.image import NonUniformImage
import matplotlib.pyplot as plt
from mpl_toolkits import mplot3d
import random as ran

wine = (p.load(open('wine.pkl', 'rb')))
```

```
#function definitions

def split_dataset(data_set):
    data_set=cp.deepcopy(data_set)
    split_set=[0]
    k=0
    p=0
    for i in range(0,len(data_set)):
        if k<5:
            if data_set['labels'][i]==0:
                split_dataset.append(data_set['labels'][i])
                k+=1
        elif k<10 and k>=5:
            if data_set['labels'][i]==1:
                split_dataset.append(data_set['labels'][i])
                k+=1
        elif k>=10 and k<15:
            if data_set['labels'][i]==2:
                split_dataset.append(data_set['labels'][i])
                k+=1
    return split_dataset

def genFitPlot(xs1,mu1,sig1,pi1,xs2,mu2,sig2,pi2):
    pi=1/3
    return plot2VarFunc(xs1,mu1,sig1,pi1,xs2,mu2,sig2,pi2)


def scalarProduct(k,vec):
    vec = cp.deepcopy(vec)
    for i in range(0,len(vec)):
        vec[i]=k*vec[i]
    return vec

def gen_function(x0,mu1,sig1,pi1,x1,mu2,sig2,pi2):
    #gaussian function form
    f_hat=20*(pi1*(1/((2*np.pi*sig1)**.5))*(np.exp((-1*((x0-
mu1)**2)/(2*(sig1**2)))))*(pi2*(1/((2*np.pi*sig2)**.5))*np.exp((-
1*((x1-mu2)**2)/(2*(sig2**2))))))

    return f_hat

def plot2VarFunc(x0,mu1,sig1,pi1,x1,mu2,sig2,pi2):
    #x = np.arange(-10, 10, 0.1)
    #y = np.arange(-10, 10, 0.1)

    X, Y = np.meshgrid(x0, x1) #need this mesh grid thing to make
surface plots
    #Z = 5*np.exp((X**2 + Y**2)/5)**-2#(np.sin(X))*(np.sin(Y))
    Z = gen_function(X,mu1,sig1,pi1,Y,mu2,sig2,pi2)
```

```
    # Set axes label
    #fig.colorbar(surf, shrink=0.5, aspect=8)

    #print(X,Y,Z)
    #plt.show()
    return (X,Y,Z)

def get_y_MargeDists(data_set):
    #print((wine['testlabels']))
    #print(len(wine['data']))
    yi_instances=[0,0,0]
    #print(wine['labels'])
    for i in range(0,len(data_set['labels'])):
        if data_set['labels'][i]==0:
            yi_instances[0]+=1
        elif data_set['labels'][i]==1:
            yi_instances[1]+=1
        elif data_set['labels'][i]==2:
            yi_instances[2]+=1
    marg_Dists=scalarProduct(1/len(data_set['labels']),yi_instances)
    #print(yi_instances)
    #print(marg_Dists)
    return marg_Dists

def getExtrimaVals(data_set,feat_index):
    input_data=[]
    input_features=[]
    for i in range(0,13): #create 13 possible features to hold
        input_features.append([])
    for i in range(0,len(data_set['labels'])): #add the feature valids
of each specified label even into the data array
        input_data.append(data_set['data'][i])
    #print(len(input_data))
    for i in range(0,len(input_data)):
        for j in range(0,len(input_data[i])):
            input_features[j].append(input_data[i][j])
    return
(np.min(input_features[feat_index]),np.max(input_features[feat_index]))

def x_given_y(data_set,y,x1,x2): #get distributions of the 13
parameters given a particular y value
    input_data=[]
    input_features=[]
    for i in range(0,13): #create 13 possible features to hold
        input_features.append([])
    for i in range(0,len(data_set['labels'])): #add the feature valids
of each specified label even into the data array
```

```
        if data_set['labels'][i]==y:#check if item in data set matches
condition
            input_data.append(data_set['data'][i])
    #print(len(input_data))
    for i in range(0,len(input_data)):
        for j in range(0,len(input_data[i])):
            input_features[j].append(input_data[i][j])
    #print(input_features[0])
    #print('mean',np.mean(input_features[0]))
    #print('var',np.var(input_features[0]))
    extrema1=getExtrimaVals(data_set,x1)
    extrema2=getExtrimaVals(data_set,x2)
    ins_outs=genFitPlot(xs1=np.arange(extrema1[0],extrema1[1],.1),mu1=n
p.mean(input_features[x1]),sig1=np.var(input_features[x1]),pi1=get_y_Ma
rgeDists(data_set)[y],
                        xs2=np.arange(extrema2[0],extrema2[1],.1),mu2=n
p.mean(input_features[x2]),sig2=np.var(input_features[x2]),pi2=get_y_Ma
rgeDists(data_set)[y])
    c=[0,0,0]
    c[y]+=1
    l=str(y)
    return ins_outs

def x_conditionals(data_set,x1,x2):
    conditionals=[]
    for i in range(0,3):
        conditionals.append(x_given_y(data_set,i,x1,x2))
    return conditionals

def closeVals(l,x0):#take in list and value and return index of value
closes to it in the list
    closeIndex=0
    bigNum=abs(l[0]-x0)
    for i in range(0,len(l)):
        if abs(x0-l[i])<bigNum:
            bigNum = abs(x0-l[i])
            closeIndex=i
    return closeIndex

def
return_predicted_z(data_set,wine_type,conds,data_item,label,feat_i_1,fe
at_i_2):
    a=0
    b=0
    q=data_item[feat_i_1]
    #print(q)
    #print(conds[0][wine_type][0])
    for j in range(0,len(conds)):
```

```
            for k in range(len(conds[j][0])):
                a = closeVals(conds[j][wine_type][0],q)
                #print(closeVals(conds[j][0][0],q))
                #print(a)
        q=data_item[feat_i_2]
        #print(q)
        #print(conds[0][1][0])
        arr=[]
        for p in conds[0][wine_type]:
            arr.append(p[0])
        #print(arr)
        for j in range(0,len(conds)):
            for k in range(len(conds[j][wine_type])):
                b = closeVals(arr,q)
        actual_wine=label
        return (conds[0][2][b][a],wine_type,actual_wine)

def getMostLikelyWine(data_set,data_item,label,feat1,feat2):
    xcs=x_conditionals(data_set,feat1,feat2)
    zs=[]
    types=[]
    truth=0
    for i in range(0,3):
        rz=return_predicted_z(wine,i,xcs,data_item,label,feat1,feat2)
        #print("predicted z is ",rz[0], "if it is wine type",rz[1])
        zs.append(rz[0])
        types.append(rz[1])
        truth=rz[2]
    f=types[zs.index(np.max(zs))]
    y=truth
    #print('f =',f, ', y =',y)
    return (f,y)

def computeErr(data_set,f1,f2):
    errors=0
    total=0
    for i in range(0,len(data_set['testlabels'])):
        g =
getMostLikelyWine(data_set,data_set['testdata'][i],data_set['testlabels
'][i],f1,f2)
        if g[0]!=g[1]:
            errors+=1
        total+=1
    return errors/total

def crossValidate():
    k=3
    errs=[]
```

```
    combos=[]
    for i in (range(0,k)):
        for j in range(0,k):
            err=computeErr(wine,i,j)
            errs.append(err)
            combos.append([i,j])
            print('error for feature combo ('+str(i)+','+str(j)+') '+'
is '+ str(err))
    print("best err is",np.min(errs),"with
combo",combos[errs.index(np.min(errs))])
    return np.min(errs)

#get_y_MargeDists(wine)
#xcs=x_conditionals(wine,0,1)
#print(np.linspace(0,10,11))
#test2DHist(xcs)
#plot2VarFunc(np.arange(-4, 4, 2),1,1,1,np.arange(0, 6, 2),1,1,1)
#genFitPlot(np.linspace(-9,9,100))
#print(split_dataset)
#x_given_y(wine,2,0,1)
#print(xcs)
#print(closeVals([.1,.2,.3,.4,.5,2.5,5,10],2))
#print("predicted z is ",return_predicted_z(wine,xcs,50,0,1))
#getMostLikelyWine(wine,wine['testdata'][50],wine['testlabels'][50],0,1
)
#print(len(wine['testlabels']))
#print(computeErr(wine))
crossValidate()
#print(ran.shuffle(wine['labels']))
#plt.show()
```

B.

The best we ended up with is the classifier features being [1,1] with a 0.47 training error rate. the parameters of this generative model for the distribution where:

$$y \rightarrow (\pi, \mu, \sigma)$$

these are the π_k, μ_k,and σ_k of the generative gaussian.

$$(\pi_0, \mu_0, \sigma_0) = (0.32, 1.90, 0.26)$$

$$(\pi_1, \mu_1, \sigma_1) = (0.39, 1.87, 0.92)$$

$$(\pi_2, \mu_2, \sigma_2) = (0.27, 3.05, 0.84)$$

```
error for feature combo (0,0)  is 0.6111111111111112
error for feature combo (0,1)  is 0.6111111111111112
error for feature combo (0,2)  is 0.6
error for feature combo (1,0)  is 0.8333333333333334
error for feature combo (1,1)  is 0.4777777777777778
error for feature combo (1,2)  is 0.7
error for feature combo (2,0)  is 0.7
error for feature combo (2,1)  is 0.7
error for feature combo (2,2)  is 0.6666666666666666
best err is 0.4777777777777778 with combo [1, 1]
0.4777777777777778
```

Question – 2

A.

It is clear to us that f^* is the optimal classifier, which, by definition, is one that makes the most probable prediction.

We know that the error rate for the optimal classifier f^* is given by:

$$err[f^*] = \int_0^1 Pr(f^*(x) \neq y|X = x)\, Pr(X = x)dx$$

Therefore,

$$err[f^*] = Pr(f^*(x) \neq 0|X = x)\, Pr(x \leq 0.2) + Pr(f^*(x) \neq 1|X = x)\, Pr(0.2 < x < 0.8) + Pr(f^*(x) \neq 0|X = x)\, Pr(x \geq 0.8)$$

$$err[f^*] = 0.25.\left(\frac{0.2}{1.0}\right) + 0.4.\left(\frac{(0.8 - 0.2)}{1.0}\right) + 0.3.\left(\frac{(1 - 0.8)}{1.0}\right)$$

$$err[f^*] = 0.05 + 0.24 + 0.06$$

$$err[f^*] = 0.05 + 0.24 + 0.06$$

$$\boldsymbol{err[f^*] = 0.35}$$

To answer Question 2, we have first provided code, and then attached output corresponding to each of the parts.

Python Code for 2A, 2B, 2C, and 2D:

```
import numpy as np
import matplotlib.pyplot as plt

# Pr(Y=1|X=x) = p0 if x <= t1
# Pr(Y=1|X=x) = p1 if t1 < x < t2
# Pr(Y=1|X=x) = p2 if x >= t2

t1 = 0.2
t2 = 0.8
p0 = 0.25
p1 = 0.6
p2 = 0.3

#########################################################################
#########
# Part (a)
err = .25*(.2)+.4*(.8-.2)+.3*(1-.8)
print('error rate for the default classifier is',err)

#########################################################################
#########
# Part (b)

def pr_y_is_1(x0):
    k=0
    if x0<=.2:
        k = .25
    elif x0>=.8:
        k = .3
    else:
        k = .6
    return k

def pr_y_is_0(x0):
    k=0
    if x0<=.2:
        k = .75
    elif x0>=.8:
        k = .7
    else:
        k = .4
    return k

# Function to implement
def stump_err(t, a, b):
    k=0
```

```
    if t<=.2:
        if a == 0:
            k = .25*(t-0)+ .25*(.2-t) +.4*(.8-.2)+.3*(1-.8)
        elif b == 0:
            k = .75*(t-0)+ .75*(.2-t) +.6*(.8-.2)+.7*(1-.8)
    elif t>=.8:
        if a == 0:
            k = .25*(.2)+.6*(.8-.2)+.7*(t-.8)+.3*(1-t)
        elif b == 0:
            k = .75*(.2)+.4*(.8-.2)+.3*(t-.8)+.7*(1-t)
    else:
        if a == 0:
            k = .25*.2+(.6*(t-.2)+.4*(.8-t))+.7*(1-.8)
        elif b==0:
            k = .75*.2+(.4*(t-.2)+.6*(.8-t))+.3*(1-.8)
        else:
            k=0
    return k

    # XXX imlement me
    return 0

#print(pr_y_is_1(.9))
print(f'T1: {stump_err(0.4,0,1)}')
print(f'T2: {stump_err(0.5,0,1)}')
print(f'T3: {stump_err(0.5,1,0)}')

#########################################################################
#########
# Part (c)

# Plotting code
t_vals = np.linspace(-0.2, 1.2, num=14001)
best_stump_err_rates = [] # XXX implement me
for x in t_vals:
    best_stump_err_rates.append(stump_err(x,0,1))
plt.figure()
plt.plot(t_vals, best_stump_err_rates)
#plt.xlim(0,1)
#plt.ylim(0,1)
plt.xlabel('$t$')
plt.ylabel('best stump error rate with predicate $x \leq t$')
plt.savefig('error_rates.pdf', bbox_inches='tight')
#plt.show()

#########################################################################
#########
# Part (d)
```

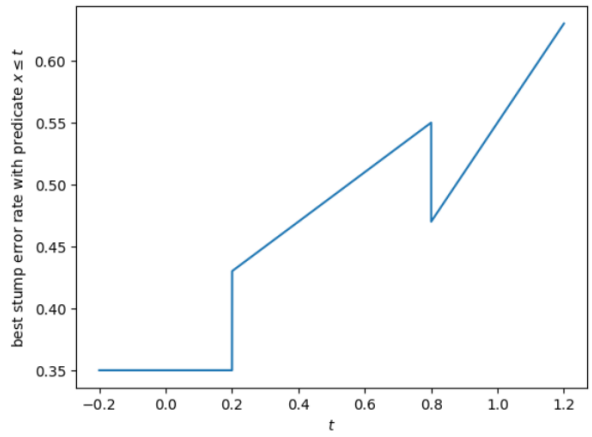
```
# Function to implement
def findErr1Stump(t,a,b,x,y):
    # XXX implement me
    err=0
    tot=0
    #print(y)
    t_index=0
    closest_threshold_t=100
    for index in range(0,len(x)):
            if abs(t-x[index])<closest_threshold_t:
                closest_threshold_t=abs(t-x[index])
                t_index=index
    t=x[t_index]

    #print(x.index(t))
    for i in range(0,len(x)):
        #print(i)
        if i<=x.index(t):
            if y[i]!=a:
                #print('made it here 1')
                err+=1
        elif i>x.index(t):
            #print('made it here 2.1','this is y=',(y[i]),)
            if y[i]!=b:
                #print('made it here 2.2')
                err+=1
        tot+=1
    #print('error #',err)
    return (err/tot,t,a,b)

def find_best_stump(x,y):
    train_err_rate =[]
    for x0 in x:
        errRate=findErr1Stump(x0,1,0,x,y)
        train_err_rate.append(errRate)
        errRate2 = findErr1Stump(x0,0,1,x,y)
        train_err_rate.append(errRate2)
    #print('training error rate: ',train_err_rate)
    t = 0
    a = 0
    b = 0
    minerr = 0
    minimum = 20.0
    for stump in train_err_rate:
        if stump[0] < minimum:
            minimum = stump[0]
```

```
    for stump in train_err_rate:
        if stump[0] == minimum:
            t = stump[1]
            a = stump[2]
            b = stump[3]
            minerr = minimum
    print('part d printout',(t, a, b, minerr))
    return (t, a, b, minerr)

x = [0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.65,
0.7, 0.75, 0.8, 0.85, 0.9]
y = [0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1]
t, a, b, train_err = find_best_stump(x, y)
#print(f'{(t,a,b)}, {train_err}')

##########################################################################
#########
# Part (e)

# Function to generate data
def generate_data(n):
    x = np.random.rand(n)
    #print(x)
    z = np.random.rand(n)
    y = np.zeros(n)
    y[x <= t1] = z[x <= t1] <= p0
    y[(x > t1) * (x < t2)] = z[(x > t1) * (x < t2)] <= p1
    y[x >= t2] = z[x >= t2] <= p2
    return (list(x),list(y))

# Simulation code

np.random.seed(42)
n = 10
num_trials = 5
error_rates = np.zeros(num_trials)
thresholds = np.zeros(num_trials)
for trial in range(num_trials):
    #print(trial)
    t, a, b, _ = find_best_stump(*generate_data(n))
    thresholds[trial] = t
    error_rates[trial] = stump_err(t, a, b)

# Plotting code
plt.figure()
plt.hist(thresholds, bins=50)
plt.xlabel('$\hat\\theta$')
plt.ylabel('counts')
```

```
plt.savefig('histogram1.pdf', bbox_inches='tight')
plt.close()

plt.figure()
plt.hist(error_rates, bins=50)
plt.xlabel('error rate')
plt.ylabel('counts')
plt.savefig('histogram2.pdf', bbox_inches='tight')
plt.close()
```

CODE OUTPUT:

```
PART A: error rate for the default classifier is 0.35000000000000003
PART B T1: 0.47
PART B T2: 0.49
PART B T3: 0.51
PART C : in this plot it can be seen that the plot is at it's minimum in the [0,.2] region and so that is the the optimal location to place the stump
```



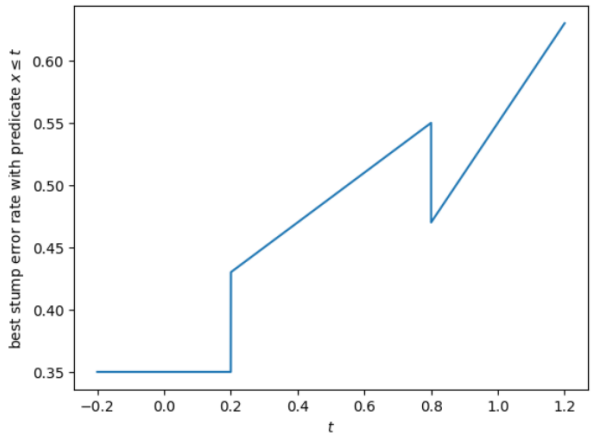

```
PART D printout (treshold, a, b, error rate) = (0.75, 0, 1, 0.17647058823529413)
```

B.

```
PART B T1: 0.47
PART B T2: 0.49
PART B T3: 0.51
```

C.

```
PART C : in this plot it can be seen that the plot is at it's minimum in the [0,.2] region and so that is the the optimal location to place the stump
```



D.

```
PART D printout (treshold, a, b, error rate) = (0.75, 0, 1, 0.17647058823529413)
```

E.

Now, putting the python code for 2E and 2F.

Python Code for 2E and 2F:

```
import numpy as np
import matplotlib.pyplot as plt

# Pr(Y=1|X=x) = p0 if x <= t1
# Pr(Y=1|X=x) = p1 if t1 < x < t2
# Pr(Y=1|X=x) = p2 if x >= t2

t1 = 0.2
t2 = 0.8
p0 = 0.25
p1 = 0.6
p2 = 0.3

#########################################################################
#########
# Part (a)
err = .25*(.2)+.4*(.8-.2)+.3*(1-.8)
#print('error rate for the default classifier is',err)

#########################################################################
#########
# Part (b)

def pr_y_is_1(x0):
    k=0
    if x0<=.2:
        k = .25
    elif x0>=.8:
        k = .3
    else:
        k = .6
    return k

def pr_y_is_0(x0):
    k=0
    if x0<=.2:
        k = .75
    elif x0>=.8:
```

```
        k = .7
    else:
        k = .4
    return k

# Function to implement
def stump_err(t, a, b):
    k=0
    if t<=.2:
        if a == 0:
            k = .25*(t-0)+ .25*(.2-t) +.4*(.8-.2)+.3*(1-.8)
        elif b == 0:
            k = .75*(t-0)+ .75*(.2-t) +.6*(.8-.2)+.7*(1-.8)
    elif t>=.8:
        if a == 0:
            k = .25*(.2)+.6*(.8-.2)+.7*(t-.8)+.3*(1-t)
        elif b == 0:
            k = .75*(.2)+.4*(.8-.2)+.3*(t-.8)+.7*(1-t)
    else:
        if a == 0:
            k = .25*.2+(.6*(t-.2)+.4*(.8-t))+.7*(1-.8)
        elif b==0:
            k = .75*.2+(.4*(t-.2)+.6*(.8-t))+.3*(1-.8)
        else:
            k=0
    return k

    # XXX imlement me
    return 0

#print(pr_y_is_1(.9))
#print(f'T1: {stump_err(0.4,0,1)}')
#print(f'T2: {stump_err(0.5,0,1)}')
#print(f'T3: {stump_err(0.5,1,0)}')

#########################################################################
#########
# Part (c)

# Plotting code
t_vals = np.linspace(-0.2, 1.2, num=14001)
best_stump_err_rates = [] # XXX implement me
for x in t_vals:
    best_stump_err_rates.append(stump_err(x,0,1))
plt.figure()
plt.plot(t_vals, best_stump_err_rates)
#plt.xlim(0,1)
#plt.ylim(0,1)
```

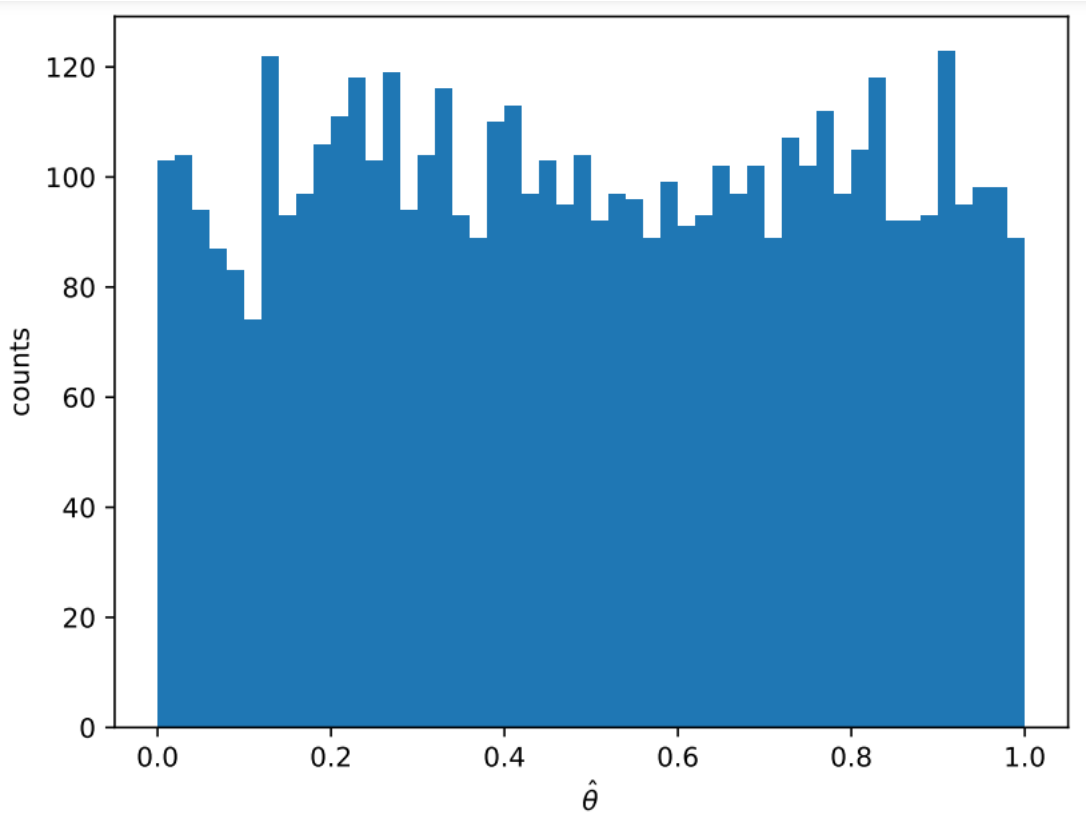
```
plt.xlabel('$t$')
plt.ylabel('best stump error rate with predicate $x \leq t$')
plt.savefig('error_rates.pdf', bbox_inches='tight')
#plt.show()

########################################################################
#########
# Part (d)

# Function to implement
def findErr1Stump(t,a,b,x,y):
    # XXX implement me
    err=0
    tot=0
    #print(y)
    t_index=0
    closest_threshold_t=100
    for index in range(0,len(x)):
            if abs(t-x[index])<closest_threshold_t:
                closest_threshold_t=abs(t-x[index])
                t_index=index
    t=x[t_index]

    #print(x.index(t))
    for i in range(0,len(x)):
        #print(i)
        if i<=x.index(t):
            if y[i]!=a:
                #print('made it here 1')
                err+=1
        elif i>x.index(t):
            #print('made it here 2.1','this is y=',(y[i]),)
            if y[i]!=b:
                #print('made it here 2.2')
                err+=1
        tot+=1
    #print('error #',err)
    return (err/tot,t,a,b)

def find_best_stump(x,y):
    train_err_rate =[]
    for x0 in x:
        errRate=findErr1Stump(x0,1,0,x,y)
        train_err_rate.append(errRate)
        errRate2 = findErr1Stump(x0,0,1,x,y)
        train_err_rate.append(errRate2)
    #print('training error rate: ',train_err_rate)
    t = 0
```

```
    a = 0
    b = 0
    minerr = 0
    minimum = 20.0
    for stump in train_err_rate:
        if stump[0] < minimum:
            minimum = stump[0]

    for stump in train_err_rate:
        if stump[0] == minimum:
            t = stump[1]
            a = stump[2]
            b = stump[3]
            minerr = minimum
    #print((t, a, b, minerr))
    return (t, a, b, minerr)

x = [0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.65,
0.7, 0.75, 0.8, 0.85, 0.9]
y = [0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1]
t, a, b, train_err = find_best_stump(x, y)
#print(f'{(t,a,b)}, {train_err}')

##########################################################################
#########
# Part (e)

# Function to generate data
def generate_data(n):
    x = np.random.rand(n)
    #print(x)
    z = np.random.rand(n)
    y = np.zeros(n)
    y[x <= t1] = z[x <= t1] <= p0
    y[(x > t1) * (x < t2)] = z[(x > t1) * (x < t2)] <= p1
    y[x >= t2] = z[x >= t2] <= p2
    return (list(x),list(y))

# Simulation code

np.random.seed(42)
n = 100
num_trials = 5000
error_rates = np.zeros(num_trials)
thresholds = np.zeros(num_trials)
for trial in range(num_trials):
    print(trial)
    t, a, b, err_rate = find_best_stump(*generate_data(n))
```

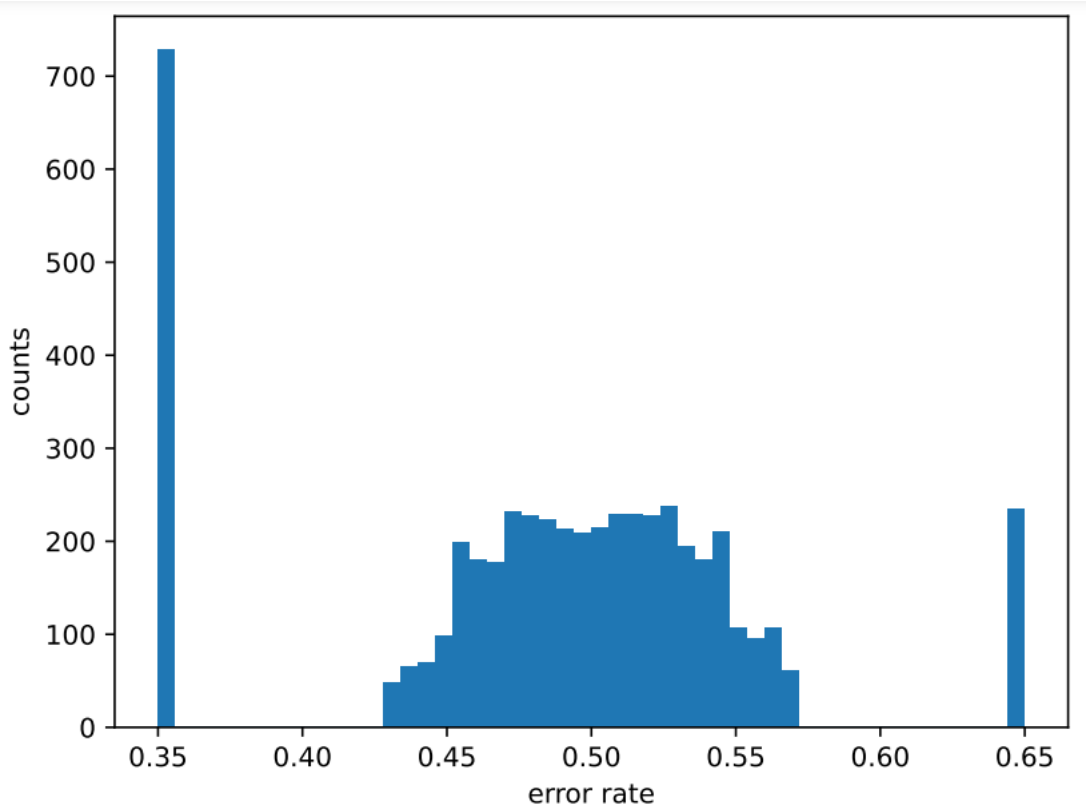
```
    thresholds[trial] = t
    error_rates[trial] = stump_err(t, a, b)

# Plotting code
plt.figure()
plt.hist(thresholds, bins=50)
plt.xlabel('$\hat\\theta$')
plt.ylabel('counts')
plt.savefig('histogram1.pdf', bbox_inches='tight')
plt.close()

plt.figure()
plt.hist(error_rates, bins=50)
plt.xlabel('error rate')
plt.ylabel('counts')
plt.savefig('histogram2.pdf', bbox_inches='tight')
plt.close()
```

Outputs:

➔ **Histogram for $\hat{t}$**



➔ **Histogram for $\mathbf{err}[\mathbf{f}_{\hat{\mathbf{T}}}]$**



F.

For the histogram of $\hat{\boldsymbol{t}}$, we see that there are four peaks at, $\hat{\theta} = 0.16$, $\hat{\theta} = 0.28$, $\hat{\theta} = 0.82$, and $\hat{\theta} = 0.92$ approximately. For the histogram of $\mathbf{err}[\mathbf{f}_{\hat{\mathbf{T}}}]$, we see that there are three peaks at, error rate $= 0.35$, error rate $= 0.50$, and error rate $= 0.65$ approximately.

For the histogram of $\mathbf{err}[\mathbf{f}_{\hat{\mathbf{T}}}]$, we expect the peaks to be centred near about eror rate = 0.50, as the labels corresponding to the samples are generated randomly, and since there are only two labels, 0 and 1, the probability of each label being generated is 50%. Thus, error rate of most decision stumps are centred around error rate = 0.50. However, it may be noted that due to the randomness of generating samples as well as the stumps, there are a few outliers around the error rate = 0.50, and therefore we also see some stumps centred around the error rate = 0.35 and error rate = 0.65, which happen to be not too far from error rate = 0.50, as some of the stumps naturally performed better than the error rate of 50 percent, and some performed worse than the error rate of 50%. But most of the stumps perform closely to an error rate of 50%.

For the histogram of $\hat{t}$**,** we can see that most of the stumps were generated in uniformly between 0 and 1. However, due to the randomness in generating samples as well as generating the stumps, we can clearly see that there exist some outliers, and therefore

slightly more number of stumps were generated around certain locations, such as at $\hat{\theta} = 0.86$ (approximately), and slightly less number of stumps were generated around certain locations, such as at $\hat{\theta} = 0.13$ (approximately).

In Part C, over the samples provided to us, we can make an inference that at certain thresholds, the minimum error rate of stumps dramatically changes, such as at t = 0.2, where it suddenly increases, and at t = 0.8, where it suddenly decreases. This observation is probably because of the randomness in the presence of the labels and positioning of stumps. Extrapolating this inference onto a much larger population provided to us in Part E, we can conclude that even though the stumps are uniformly generated between 0 and 1, there are certain outlier locations, where a greater number of stumps are concentrated. Similarly, due to the randomness in the generated samples and stumps, certain stumps have a greater error rate than 50%, and some have less than that.

Question – 3

A.

Using the sci-kit learn library of python, LinearRegression is deployed to fit a linear model over the each of the individual eight features to the lpsa in the prostate dataset. The Linear Regression model created using Sci-Kit learn library minimizes the residual sum of squares (or sum of squared errors) between the observed targets in dataset to the targets predicted by linear approximation.

Python Code:

```
import numpy as np
lcavol = []
lweight = []
age = []
lbph = []
svi = []
lcp = []
gleason = []
pgg45 = []

lpsa = []

for i in range(len(prostate['data'])):
  lcavol.append(prostate['data'][i][0])
  lpsa.append(prostate['data'][i][8])
  lweight.append(prostate['data'][i][1])
  age.append(prostate['data'][i][2])
  lbph.append(prostate['data'][i][3])
```

```
  svi.append(prostate['data'][i][4])
  lcp.append(prostate['data'][i][5])
  gleason.append(prostate['data'][i][6])
  pgg45.append(prostate['data'][i][7])

lcavol = np.array(lcavol)
lpsa = np.array(lpsa)
lweight = np.array(lweight)
age = np.array(age)
lbph = np.array(lbph)
svi = np.array(svi)
lcp = np.array(lcp)
gleason = np.array(gleason)
pgg45 = np.array(pgg45)

#print(len(lcavol))
#print(len(lpsa))

# creating a linear regression model for the features lcavol and lpsa
from sklearn.linear_model import LinearRegression

lcavol = lcavol.reshape(-1,1)
lweight = lweight.reshape(-1,1)
age = age.reshape(-1,1)
lbph = lbph.reshape(-1,1)
svi = svi.reshape(-1,1)
lcp = lcp.reshape(-1,1)
gleason = gleason.reshape(-1,1)
pgg45 = pgg45.reshape(-1,1)

lm = LinearRegression()
lm.fit(lcavol,lpsa)
print("intercept of best fit affine function for lcavol: ",
round(lm.intercept_,3))
print("slope of best fit affine function for lcavol: ",
round(lm.coef_[0],3))

lm = LinearRegression()
lm.fit(lweight,lpsa)
print("intercept of best fit affine function for lweight: ",
round(lm.intercept_,3))
print("slope of best fit affine function for lweight: ",
round(lm.coef_[0],3))

lm = LinearRegression()
lm.fit(age,lpsa)
print("intercept of best fit affine function for age: ",
round(lm.intercept_,3))
```

```
print("slope of best fit affine function for age: ",
round(lm.coef_[0],3))

lm = LinearRegression()
lm.fit(lbph,lpsa)
print("intercept of best fit affine function for lbph: ",
round(lm.intercept_,3))
print("slope of best fit affine function for lbph: ",
round(lm.coef_[0],3))


lm = LinearRegression()
lm.fit(svi,lpsa)
print("intercept of best fit affine function for svi: ",
round(lm.intercept_,3))
print("slope of best fit affine function for svi: ",
round(lm.coef_[0],3))

lm = LinearRegression()
lm.fit(lcp,lpsa)
print("intercept of best fit affine function for lcp: ",
round(lm.intercept_,3))
print("slope of best fit affine function for lcp: ",
round(lm.coef_[0],3))

lm = LinearRegression()
lm.fit(gleason,lpsa)
print("intercept of best fit affine function for gleason: ",
round(lm.intercept_,3))
print("slope of best fit affine function for gleason: ",
round(lm.coef_[0],3))

lm = LinearRegression()
lm.fit(pgg45,lpsa)
print("intercept of best fit affine function for pgg45: ",
round(lm.intercept_,3))
print("slope of best fit affine function for pgg45: ",
round(lm.coef_[0],3))
```

Output:

```
intercept of best fit affine function for lcavol:  1.516
slope of best fit affine function for lcavol:  0.713
intercept of best fit affine function for lweight:  -2.006
slope of best fit affine function for lweight:  1.23
intercept of best fit affine function for age:  0.079
slope of best fit affine function for age:  0.037
intercept of best fit affine function for lbph:  2.437
slope of best fit affine function for lbph:  0.217
intercept of best fit affine function for svi:  2.094
slope of best fit affine function for svi:  1.602
intercept of best fit affine function for lcp:  2.543
slope of best fit affine function for lcp:  0.422
intercept of best fit affine function for gleason:  -1.475
slope of best fit affine function for gleason:  0.583
intercept of best fit affine function for pgg45:  1.967
slope of best fit affine function for pgg45:  0.018
```

B.

For the second part, instead of each of the individual features, we deploy Linear Regression to fit a linear model over all of the eight features together. So, the Linear Regression model created using Sci-kit learn library minimizes the residual sum of squares (or sum of squared errors) between the observed targets in the dataset to the targets predicted with linear approximation.

Python Code:

```
eight_features = []
for i in range(len(prostate['data'])):
  eight_features.append(prostate['data'][i])
  eight_features[i] = eight_features[i][0:8]

eight_features = np.array(eight_features)
#print(eight_features)
#print(len(eight_features))
#print(len(eight_features[0]))
lm = LinearRegression()
lm.fit(eight_features, lpsa)
print("Intercept of best fit affine function with all eight features:
", round(lm.intercept_,3))
for i in range(len(lm.coef_)):
  lm.coef_[i] = round(lm.coef_[i], 3)
print("coefficients of best fit affine function with all eight
features: ", lm.coef_)
print("slope for feature lcavol of best fit affine function with all
eight features: ", lm.coef_[0])
```

```
print("slope for feature lweight of best fit affine function with all
eight features: ", lm.coef_[1])
print("slope for feature age of best fit affine function with all eight
features: ", lm.coef_[2])
print("slope for feature lbph of best fit affine function with all
eight features: ", lm.coef_[3])
print("slope for feature svi of best fit affine function with all eight
features: ", lm.coef_[4])
print("slope for feature lcp of best fit affine function with all eight
features: ", lm.coef_[5])
print("slope for feature gleason of best fit affine function with all
eight features: ", lm.coef_[6])
print("slope for feature pgg45 of best fit affine function with all
eight features: ", lm.coef_[7])
```

Output:

```
Intercept of best fit affine function with all eight features:  0.429
coefficients of best fit affine function with all eight features:  [ 0.577  0.614 -0.019  0.145  0.737 -0.206 -0.03   0.009]
slope for feature lcavol of best fit affine function with all eight features:  0.577
slope for feature lweight of best fit affine function with all eight features:  0.614
slope for feature age of best fit affine function with all eight features:  -0.019
slope for feature lbph of best fit affine function with all eight features:  0.145
slope for feature svi of best fit affine function with all eight features:  0.737
slope for feature lcp of best fit affine function with all eight features:  -0.206
slope for feature gleason of best fit affine function with all eight features:  -0.03
slope for feature pgg45 of best fit affine function with all eight features:  0.009
```

Question – 4

A.

In accordance with the Normal Linear Regression model, we can conclude since X = (X_1, X_2) is a bivariate normally distributed random variable, the conditional distribution of Y given X = X_1, is given by:

$$N({w_1}^T X_1, \sigma_1^2)$$

$$w_1 - weight\ coefficient\ vector$$

Now, we know that the smallest Mean Squared Error possible is σ_1^2. The affine function of X_1 that has the smallest possible MSE is given by: $w^T X_1$.

Mathematically,

For the best possible prediction:

$$E[Y|X = X_1] = w^T X_1$$

So, generally,

$$m = \frac{avg(xy) - avg(x)avg(y)}{avg(x^2) - \big(avg(x)\big)^2} \quad (1)$$

$$b = avg(y) - m\, avg(x) \quad (2)$$

Replacing the averages in (1) and (2) with the Expectations with respect to the distribution of (X, Y).

For the random variable X_1, we obtain the slope and intercept of the affine function as the following:

$$m = \frac{E(X_1 Y) - E(X_1)E(Y)}{E(X_1^2) - \big(E(X_1)\big)^2} \quad (3)$$

$$b = \mathrm{E(Y)} - \mathrm{m\ E(X_1)} \quad (4)$$

Given that,

$$Y = \frac{3}{2}X_1 - \frac{3}{4}X_2 \quad (5)$$

Substituting (5) in (3) to obtain a derived expression for slope,

$$m = \frac{E\left(X_1(\frac{3}{2}X_1 - \frac{3}{4}X_2)\ \right) - E(X_1)E\left(\frac{3}{2}X_1 - \frac{3}{4}X_2\right)}{E(X_1^2) - \big(E(X_1)\big)^2} \quad (6)$$

Further solving (6), we obtain,

$$m = \frac{\frac{3}{2}E(X_1^2) - \frac{3}{4}E(X_1X_2) - \frac{3}{2}\big(E(X_1)\big)^2 + \frac{3}{4}E(X_1)E(X_2)}{E(X_1^2) - \big(E(X_1)\big)^2} \quad (7)$$

$$m = \frac{\frac{3}{2}\Big(E(X_1^2) - \big(E(X_1)\big)^2\Big) - \frac{3}{4}E(X_1X_2) + \frac{3}{4}E(X_1)E(X_2)}{E(X_1^2) - \big(E(X_1)\big)^2} \quad (8)$$

We know that $var(X_1) = 1$,

$$E(X_1^2) - \big(E(X_1)\big)^2 = 1 \quad (9)$$

Substituting (9) into (8) for further solving,

$$m = \frac{\frac{3}{2}E(X_1^2) - \frac{3}{4}E(X_1X_2) - \frac{3}{2}\big(E(X_1)\big)^2 + \frac{3}{4}E(X_1)E(X_2)}{E(X_1^2) - \big(E(X_1)\big)^2}$$

$$m = \frac{3}{2} - \frac{3}{4}E(X_1X_2) + \frac{3}{4}E(X_1)E(X_2) \tag{10}$$

$$m = \frac{3}{2} - \frac{3}{4}\big(E(X_1X_2) - E(X_1)E(X_2)\big) \tag{11}$$

We also know that,

$$cov(X_1, X_2) = E(X_1X_2) - E(X_1)E(X_2) \tag{12}$$

$$E(X_1X_2) - E(X_1)E(X_2) = \frac{2}{3} \tag{13}$$

Substituting (13) into (11) to obtain the slope,

$$m = \frac{3}{2} - \frac{3}{4}.\frac{2}{3}$$

$$m = 1 \tag{14}$$

Now, we have obtained the value of slope, so we will substitute (14) into (4) to obtain the intercept,

$$b = \mathrm{E(Y)} - (1)\mathrm{E(X_1)}$$

$$b = \mathrm{E(Y)} - \mathrm{E(X_1)} \tag{15}$$

Taking expectation on both sides of (5),

$$E\,(Y) = \frac{3}{2}\,E(X_1) - \frac{3}{4}\,E(X_2) \tag{16}$$

Putting (16) in (15) for further solving,

$$b = \frac{3}{2}\,E(X_1) - \frac{3}{4}\,E(X_2) - \mathrm{E(X_1)}$$

As both X_1 and X_2 have zero mean, as stated in the question,

$$E(X_1) = E(X_2) = 0$$

$$\Rightarrow b = 0 \tag{17}$$

Using (14) and (17), slope and intercept are as follows:

$$\boldsymbol{m = 1, b = 0}$$

B.

In accordance with the Normal Linear Regression model, we can conclude since X = (X_1, X_2) is a bivariate normally distributed random variable, the conditional distribution of Y given X = X_1, is given by:

$$N({w_2}^T X_2, \sigma_2^2)$$

$$w_2 - weight\ coefficient\ vector$$

Now, we know that the smallest Mean Squared Error possible is σ_2^2. The affine function of X_1 that has the smallest possible MSE is given by: $w_2^T X_2$.

Mathematically,

For the best possible prediction:

$$E[Y|X = X_2] = w^T X_2$$

So, generally,

$$m = \frac{avg(xy) - avg(x)avg(y)}{avg(x^2) - \big(avg(x)\big)^2} \quad (1)$$

$$b = avg(y) - m\ avg(x) \quad (2)$$

Replacing the averages in (1) and (2) with the Expectations with respect to the distribution of (X, Y).

For the random variable X_2, we obtain the slope and intercept of the affine function as the following:

$$m = \frac{E(X_2 Y) - E(X_2)E(Y)}{E(X_2^2) - \big(E(X_2)\big)^2} \quad (3)$$

$$b = \mathrm{E(Y)} - \mathrm{m\ E(X_2)} \quad (4)$$

Given that,

$$Y = \frac{3}{2}X_1 - \frac{3}{4}X_2 \quad (5)$$

Substituting (5) in (3) to obtain a derived expression for slope,

$$m = \frac{E\left(X_2(\frac{3}{2}X_1 - \frac{3}{4}X_2)\ \right) - E(X_2)E\left(\frac{3}{2}X_1 - \frac{3}{4}X_2\right)}{E(X_2^2) - \left(E(X_2)\right)^2} \quad (6)$$

Further solving (6), we obtain,

$$m = \frac{\frac{3}{2}E(X_1X_2) - \frac{3}{4}E(X_2^2) - \frac{3}{2}\ E(X_1)E(X_2) + \frac{3}{4}\left(E(X_2)\right)^2}{E(X_2^2) - \left(E(X_2)\right)^2} \quad (7)$$

$$m = \frac{\frac{3}{2}E(X_1X_2) - \frac{3}{2}E(X_1)E(X_2) - \frac{3}{4}\left(E(X_2^2) - \left(E(X_2)\right)^2\right)}{E(X_2^2) - \left(E(X_2)\right)^2} \quad (8)$$

We know that $var(X_2) = 1$,

$$E(X_2^2) - \left(E(X_2)\right)^2 = 1 \quad (9)$$

Substituting (9) into (8) for further solving,

$$m = \frac{\frac{3}{2}E(X_1X_2) - \frac{3}{2}E(X_1)E(X_2) - \frac{3}{4}\left(E(X_2^2) - \left(E(X_2)\right)^2\right)}{E(X_2^2) - \left(E(X_2)\right)^2}$$

$$m = \frac{3}{2}E(X_1X_2) - \frac{3}{2}E(X_1)E(X_2) - \frac{3}{4} \quad (10)$$

$$m = \frac{3}{2}\left(E(X_1X_2) - E(X_1)E(X_2)\right) - \frac{3}{4} \quad (11)$$

We also know that,

$$cov(X_1, X_2) = E(X_1X_2) - E(X_1)E(X_2) \quad (12)$$

$$E(X_1X_2) - E(X_1)E(X_2) = \frac{2}{3} \quad (13)$$

Substituting (13) into (11) to obtain the slope,

$$m = \frac{3}{2}.\frac{2}{3} - \frac{3}{4}$$

$$m = \frac{1}{4} \quad (14)$$

Now, we have obtained the value of slope, so we will substitute (14) into (4) to obtain the intercept,

$$b = \mathrm{E(Y)} - \left(\frac{1}{4}\right)\mathrm{E}(\mathrm{X}_2) \quad (15)$$

Taking expectation on both sides of (5),

$$E\ (Y) = \frac{3}{2}\ E(X_1) - \frac{3}{4}\ E(X_2) \tag{16}$$

Putting (16) in (15) for further solving,

$$b = \frac{3}{2}\ E(X_1) - \frac{3}{4}\ E(X_2) - \frac{1}{4}\ E(X_2)$$

As both X_1 and X_2 have zero mean, as stated in the question,

$$E(X_1)\ =\ E(X_2)\ =\ 0$$

$$\Rightarrow\ b\ = 0 \tag{17}$$

Using (14) and (17), slope and intercept are as follows:

$$\boldsymbol{m = \frac{1}{4}, b = 0}$$

C.

In accordance with the Normal Linear Regression model, we can conclude since X = (X_1, X_2) is a bivariate normally distributed random variable, the conditional distribution of Y given $X\ =\ (X1, X2)$ is given by:

$$N(\mathrm{w}^T X, \sigma^2)$$

$$\mathrm{w} - weight\ coefficient\ vector$$

Now, we know that the smallest Mean Squared Error possible is σ_1^2. The affine function of X_1 that has the smallest possible MSE is given by: $w^T X$.

Mathematically,

For the best possible prediction:

$$E[Y|X\ =\ (X1, X2)] = w^T X$$

So, generally,

$$m = \frac{avg(xy) - avg(x)avg(y)}{avg(x^2) - \big(avg(x)\big)^2} \tag{1}$$

$$b = avg(y) - m\ avg(x) \tag{2}$$

Replacing the averages in (1) and (2) with the Expectations with respect to the distribution of (X, Y).

For the random variable X_2, we obtain the slope and intercept of the affine function as the following:

$$m = \frac{E(XY) - E(X)E(Y)}{E(X^2) - \big(E(X)\big)^2} \tag{3}$$

$$b = \mathrm{E(Y)} - \mathrm{m\ E}(X) \tag{4}$$

So, we know that,

$$var(X)\ =\ E(X^2) - \big(E(X)\big)^2 \tag{5}$$

Also, for the bivariate random variable, $X = (X1, X2)$

$$var(x) = \begin{pmatrix} var(X_1) & cov(X_1, X_2) \\ cov(X_2, X_1) & var(X_2) \end{pmatrix} \tag{5}$$

$$m = \frac{E(XY) - E(X)E(Y)}{var(X)} \tag{6}$$

$$m = \big(var(x)\big)^{-1}\big(E(XY) - E(X)E(Y)\big) \tag{7}$$

Now,

$$\big(var(x)\big)^{-1} = \frac{adj\big(var(x)\big)}{|var(x)|} \tag{8}$$

Given,

$$var(x) = \begin{pmatrix} 1 & \frac{2}{3} \\ \frac{2}{3} & 1 \end{pmatrix} \tag{9}$$

$$\big(var(x)\big)^{-1} = \frac{adj\left(\begin{pmatrix} 1 & \frac{2}{3} \\ \frac{2}{3} & 1 \end{pmatrix}\right)}{\left|\begin{pmatrix} 1 & \frac{2}{3} \\ \frac{2}{3} & 1 \end{pmatrix}\right|}$$

$$\big(var(x)\big)^{-1} = \frac{\begin{pmatrix} 1 & -\frac{2}{3} \\ -\frac{2}{3} & 1 \end{pmatrix}}{\frac{5}{9}}$$

$$\big(var(x)\big)^{-1} = \begin{pmatrix} \frac{9}{5} & -\frac{6}{5} \\ -\frac{6}{5} & \frac{9}{5} \end{pmatrix} \tag{10}$$

From (7),

$$m = \big(var(x)\big)^{-1}\big(E(XY) - E(X)E(Y)\big)$$

Thus,

$$m = \begin{pmatrix} \frac{9}{5} & -\frac{6}{5} \\ -\frac{6}{5} & \frac{9}{5} \end{pmatrix} \begin{pmatrix} E(X_1Y) - E(X_1)E(Y) \\ E(X_2Y) - E(X_2)E(Y) \end{pmatrix} \tag{11}$$

Given that,

$$Y = \frac{3}{2}X_1 - \frac{3}{4}X_2 \tag{12}$$

Substituting (12) into (11),

$$m = \begin{pmatrix} \frac{9}{5} & -\frac{6}{5} \\ -\frac{6}{5} & \frac{9}{5} \end{pmatrix} \begin{pmatrix} E\left(X_1\left(\frac{3}{2}X_1 - \frac{3}{4}X_2\right)\right) - E(X_1)E\left(\frac{3}{2}X_1 - \frac{3}{4}X_2\right) \\ E\left(X_2\left(\frac{3}{2}X_1 - \frac{3}{4}X_2\right)\right) - E(X_2)E\left(\frac{3}{2}X_1 - \frac{3}{4}X_2\right) \end{pmatrix}$$

$$\Rightarrow\ m = \begin{pmatrix} \frac{9}{5} & -\frac{6}{5} \\ -\frac{6}{5} & \frac{9}{5} \end{pmatrix} \begin{pmatrix} \frac{3}{2}\Big(E(X_1^2) - \big(E(X_1)\big)^2\Big) - \frac{3}{4}\big(E(X_1X_2) - E(X_1)E(X_2)\big) \\ \frac{3}{2}\big(E(X_1X_2) - E(X_1)E(X_2)\big) - \frac{3}{4}\Big(E(X_2^2) - \big(E(X_2)\big)^2\Big) \end{pmatrix} \tag{13}$$

We know that $var(X_1) = 1$,

$$E(X_1^2) - \big(E(X_1)\big)^2 = 1 \tag{14}$$

We also know that $var(X_2) = 1$,

$$E(X_2^2) - \big(E(X_2)\big)^2 = 1 \tag{15}$$

Alongside that,

$$cov(X_1, X_2) = E(X_1X_2) - E(X_1)E(X_2) \tag{16}$$

$$E(X_1X_2) - E(X_1)E(X_2) = \frac{2}{3} \tag{17}$$

Substituting (14), (15) and (17) into (13),

$$m = \begin{pmatrix} \frac{9}{5} & -\frac{6}{5} \\ -\frac{6}{5} & \frac{9}{5} \end{pmatrix} \begin{pmatrix} \frac{3}{2}.1 - \frac{3}{4}.\frac{2}{3} \\ \frac{3}{2}.\frac{2}{3} - \frac{3}{4}.1 \end{pmatrix}$$

$$\Rightarrow\ m = \begin{pmatrix} \frac{9}{5} & -\frac{6}{5} \\ -\frac{6}{5} & \frac{9}{5} \end{pmatrix} \begin{pmatrix} \frac{3}{2} - \frac{1}{2} \\ 1 - \frac{3}{4} \end{pmatrix}$$

$$\Rightarrow\ m = \begin{pmatrix} \frac{9}{5} & -\frac{6}{5} \\ -\frac{6}{5} & \frac{9}{5} \end{pmatrix} \begin{pmatrix} 1 \\ \frac{1}{4} \end{pmatrix}$$

Finally,

$$\Rightarrow m = \begin{pmatrix} \frac{3}{2} \\ -\frac{3}{4} \end{pmatrix} \tag{18}$$

Now, calculating the intercept,

From (4),

$b = E(\mathrm{Y}) - m\,E(X)$

Given that,

$$Y = \frac{3}{2}X_1 - \frac{3}{4}X_2$$

$$b = \frac{3}{2}E(X_1) - \frac{3}{4}E(X_2) - mE(X)$$

$$\Rightarrow b = \frac{3}{2}E(X_1) - \frac{3}{4}E(X_2) - \begin{pmatrix} m_1 \\ m_2 \end{pmatrix} (E(X_1) \quad E(X_2))$$

$$\Rightarrow\ b\ = \frac{3}{2}E(X_1) - \frac{3}{4}E(X_2) - \left(m_1E(X_1) + m_2E(X_2)\right)$$

From (18),

$$m_1 = \frac{3}{2} \tag{19}$$

$$m_2 = -\frac{3}{4} \tag{20}$$

$$\Rightarrow\ b\ = \frac{3}{2}E(X_1) - \frac{3}{4}E(X_2) - \left(\frac{3}{2}E(X_1) - \frac{3}{4}E(X_2)\right)$$

$$\Rightarrow\ b\ = \frac{3}{2}E(X_1) - \frac{3}{4}E(X_2) - \left(\frac{3}{2}E(X_1) - \frac{3}{4}E(X_2)\right)$$

$$\Rightarrow\ b\ = 0 \tag{21}$$

Thus, using (18), (19), (20), and (21), we can conclude that:

$$\boldsymbol{m} = \begin{pmatrix} \frac{3}{2} \\ -\frac{3}{4} \end{pmatrix}, \qquad \boldsymbol{m_1} = \frac{3}{2},\ \boldsymbol{m_2} = -\frac{3}{4}, \qquad \boldsymbol{b}\ = 0$$

Therefore, it can clearly be seen that the best affine predictor of Y that considers both X_1 and X_2 has a positive coefficient for X_1 and a negative coefficient for X_2.

Also, it can be seen that the best affine predictors of Y that consider X_1 and X_2 individually have positive coefficients for the variable, therefore implying that X_1 and X_2 are positively correlated with Y.